



Lab course Automatic Control II
Prof. Dr.-Ing. Knut Graichen
Chair of Automatic Control



Lab course Automatic Control II

Flatness-based control of a laboratory crane

Supervisor: Daniel Landgraf

1 Introduction

In the experiment *flatness-based control of a laboratory crane*, the transfer of a load is simulated with an experimental setup. Gantry cranes are often used in goods handling areas and assembly halls to move loads horizontally and vertically at the same time. During positioning operations, the weak self-damping of the crane can cause oscillations of the load, which delays further processing. In the worst case, this can also lead to serious accidents. To ensure the greatest possible safety and maximize throughput, the load should reach the target position as quickly as possible and with minimum oscillation. This is made possible via highly accurate position control.

Part of the practical course is the modeling of the crane in the form of nonlinear differential equations as well as the control using flatness-based methods from nonlinear control engineering. First, the flatness of the system is proven and the flatness parameterization is formulated. In order to enable a defined displacement of the load between two points as well as to avoid obstacles if necessary, a corresponding planning of desired trajectories is necessary. Based on the trajectory planning, a flatness-based feedforward control has to be designed subsequently. Due to friction and non-modeled effects, the load will not be able to follow the desired trajectory without a feedback of the system state. For this reason, flatness-based tracking controls using quasi-static and dynamic feedback linearization will be designed and their behavior will be compared. The developed components of the control system will first be tested simulatively in MATLAB/Simulink and then validated at the experimental setup. For this purpose, a real-time capable dSPACE system is used, which is also programmed via MATLAB/Simulink.

For the execution of this experiment, the contents of the lectures *Nonlinear Control Systems* are assumed. If necessary, re-read the most important topics with the help of the lecture notes [1] and [2].

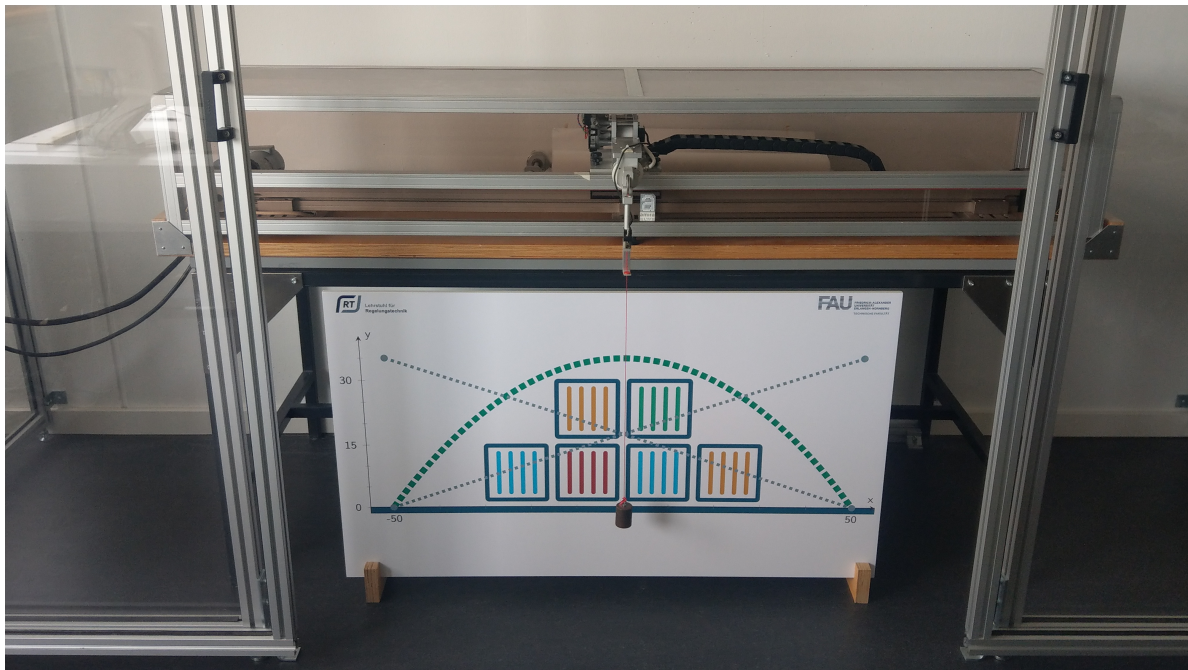


Figure 1: Experimental setup of the laboratory crane.

2 Preparation

The basis for the design of a flatness-based control is always a model of the system that should be controlled. Therefore, in a first step, a physical model of the laboratory crane is derived using the *Lagrange formalism*. The geometric relationships of the crane as well as the coordinate system chosen for modeling are shown schematically in Figure 2.

2.1 Modeling

The cart has the mass m_c and can be moved horizontally on a guide rail driven by the force F_c . In addition, the friction force F_f acts on the cart. The cart is equipped with a drum motor of radius r_m , to which a load of mass m is attached by a rope. The rope with length ℓ is assumed to be massless and always taut. The motor has a moment of inertia J_m and can be driven by the torque M_m , which allows the rope to be retracted or extended. In this case, the relation

$$\dot{\ell} = \omega_m r_m \quad (1)$$

applies. The motor is also subject to friction, which is described by the frictional torque M_f . The motor is mounted so that the load can rotate freely around the z -axis. The cart and the load are modeled as point masses in the following.

To be able to derive the equations of motion using the Lagrange formalism, first the *Lagrange function*

$$\mathcal{L} = \mathcal{T} - \mathcal{V}, \quad (2)$$

which is composed of the kinetic and potential energy $\mathcal{T}$ and $\mathcal{V}$ of the system, can be defined. $\mathcal{T}$ and $\mathcal{V}$ are thereby described depending on so-called *generalized coordinates* $\mathbf{q}$, which are chosen to be $\mathbf{q} = [q_1, q_2, q_3]^T = [x_c, \ell, \theta]^T$ for the crane. The kinetic energy $\mathcal{T}$ is composed of the kinetic energy of the cart and load and the rotational energy of the motor. The potential energy $\mathcal{V}$ of the system is equal to the potential energy of the load, since the potential energy of the cart becomes 0 due to the chosen coordination system ($y_c = 0$). Thus, the equations of motion of the crane can be calculated using the *Euler-Lagrange equations*

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_i} - \frac{\partial \mathcal{L}}{\partial q_i} = \tau_i, \quad i = 1, 2, 3. \quad (3)$$

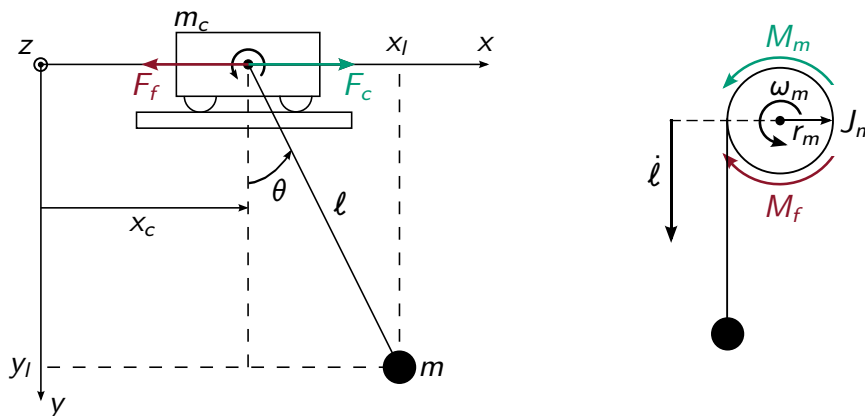


Figure 2: Geometric relations of the laboratory crane and the coordinate system chosen for modeling.

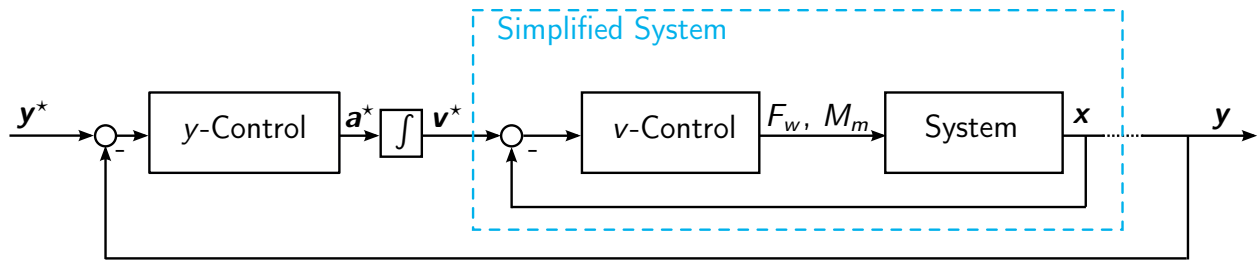


Figure 3: Schematic illustration of the cascaded control loop structure at the experimental setup.

Here, the vector $\tau = [F_{x_c}, F_\ell, 0]^T$ denotes the non-conservative forces in the direction of the generalized coordinates. In the case of the crane, these forces are composed by the forces generated by the electric motors mounted to the cart and the winch, as well as the frictional forces.

(P1) Using (3), calculate the equations of motion of the crane.

Note: You can also perform the calculation using the Symbolic Math Toolbox of MATLAB.

The control structure at the experimental setup consists of the cascaded structure shown in Figure 3 with fast inner velocity controllers for the cart and rope velocities for friction compensation. Assuming that these inner velocity controllers are sufficiently fast and precise, the dynamics can be simplified to

$$\ddot{x}_c = a_c, \quad \ddot{\ell} = a_\ell, \quad \ddot{\theta} = -\frac{1}{\ell} \left(g \sin(\theta) + a_c \cos(\theta) + 2\dot{\ell}\dot{\theta} \right) \quad (4)$$

using the cart and rope acceleration a_c and a_ℓ . In this laboratory experiment, a flatness-based control is developed based on the simplified model (4). For this purpose, the dynamics must be transformed into a state-space representation of the form $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$.

(P2) Transform the simplified dynamics (4) to the state space representation

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) \quad (5)$$

using the state and control input vectors $\mathbf{x} = [x_c, \dot{x}_c, \ell, \dot{\ell}, \theta, \dot{\theta}]^T$ and $\mathbf{u} = [a_c, a_\ell]^T$.

(P3) Name and describe an alternative method to compensate for the friction effects.

2.2 Flatness parameterization and feedforward control

For flat systems, all state and control inputs can be represented as a function of the flat output $\mathbf{z}$ and its time derivatives. By the so-called *inverse system* it is possible to calculate the required control inputs, for example to follow a desired trajectory. In this way, flatness-based feedforward control can be implemented. In the following, the flat output as well as the differential parameterization of the state and control inputs are determined for the simplified system of the laboratory crane (5).

The movement of the load in the (x, y) plane represents the controlled variable in this experiment. Therefore, the position of the load

$$\mathbf{z} = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} x_l \\ y_l \end{pmatrix} = \begin{pmatrix} x_c + \ell \sin \theta \\ \ell \cos \theta \end{pmatrix} \quad (6)$$

is chosen as a candidate for the flat output.

- (P4) Show that the position $\mathbf{z}$ of the load mass m is a flat output of the system (5) and calculate the differential parameterization of the state and control inputs.

Hint: First determine the parameterization of the angle θ via $\ddot{\mathbf{z}}$ using the relation $\sin^2 a + \cos^2 a = 1$ and then determine the states x_c and ℓ . Finally, determine the remaining states and control inputs in the simplest possible way.

- (P5) State the resulting flatness-based feedforward control for an arbitrary trajectory $\mathbf{z}^*(t)$. What differentiation class $\mathcal{C}^k$ must the trajectories satisfy for the system to follow them exactly in the nominal case?

2.3 Trajectory planning

A flatness-based feedforward control requires the planning of a desired trajectory for the two flat outputs $z_1 = x_l$ and $z_2 = y_l$. The control signals $u_1 = a_c$ and $u_2 = a_\ell$ can then be calculated from the planned trajectory and its time derivatives via the flatness parameterization. First, a polynomial trajectory for a setpoint change in the time interval $t \in [0, T]$ is planned.

- (P6) Which boundary conditions must be satisfied by the desired trajectory? Determine the polynomials $x_l(t)$ and $y_l(t)$ of lowest possible order as a function of T , the initial values $x_l(0)$ and $y_l(0)$, and the final values $x_l(T)$ and $y_l(T)$.
- (P7) Calculate all time derivatives of the desired trajectory required for flatness-based feedforward control.

In addition, a parabolic trajectory is to be planned, as it can be seen in Figure 1. The start and end points shall have the coordinates $(-0.5, 0.72)$ and $(0.5, 0.72)$, respectively. The vertex of the parabola shall be at $(0, 0.35)$.

- (P8) Find the parameters of the function $y_l(t) = f(x_l(t)) = a_2 x_l^2(t) + a_1 x_l(t) + a_0$, such that the above points lie on the parabola. Calculate $y_l(t)$ explicitly for the case where $x_l(t)$ is chosen as the polynomial of lowest possible order.
- (P9) Calculate all time derivatives of the parabolic desired trajectory required for flatness-based feedforward control.

(P10) *Implement a target trajectory generator for the setpoint change and for the parabolic trajectory as a Matlab function in Simulink. For this purpose, use the template in 'Simulation.slx'. It should be possible to specify the parameters, such as the start and end points, using the 'Initialization_Simulation.m' script. Enter there the parameters of the parabola from (P8).*

Hint: Implement the desired trajectory generators as computationally efficient as possible so that they can be executed in real time on the experimental setup.

A third desired trajectory is shall be designed, which consists of a sequence of setpoint changes. For this purpose, the point $(x_l, y_l) = (-0.5, 0.72)$ is approached from an arbitrary starting position. Then the load shall pass successively through the points $(-0.5, 0.37)$, $(0.5, 0.72)$, $(0.5, 0.37)$ and $(-0.5, 0.72)$. Finally, the load is moved back to the start position. The duration T_{Task} between the starts of two setpoint changes should be able to be specified as a parameter. The duration T of a single setpoint change should be the same for all setpoint changes.

(P11) *Implement a desired trajectory generator for the sequence of setpoint changes.*

An input signal can be generated from the planned trajectories for the flat outputs using feedforward control, so that the system shows the desired behavior in the disturbance-free case. With the help of the feedforward control, the target trajectory generators can be tested simulatively. Two simulation models are available in the Simulink model for this purpose: A kinematic model based on the simplified dynamics (4) and a reference model that simulates additional friction forces. In the following, the flatness-based feedforward control will be tested simulatively.

(P12) *Implement the feedforward control from task (P5) as a Matlab function in Simulink. Use the template in Simulation.slx.*

(P13) *Run the 'Initialization-Simulation.m' script and simulate the system with feedforward control for the three designed target trajectory generators. Use the kinematic simulation model for this purpose and select T and T_{Task} suitably. Save the plots of the actual and target positions of the load.*

(P14) *Repeat (P13), but use the reference model for simulation. What differences can you see?*

2.4 Design of a flatness-based tracking control

Due to model inaccuracies and disturbances, a stabilizing controller is necessary to ensure accurate movement of the load along the desired trajectory. In the following, a *flatness-based tracking control* is designed for the system. With the help of such a control, it is possible to cover larger operating ranges where the assumption of a linearized model around the operating point is no longer sufficient. Since the sum of the highest output derivatives in the flatness parameterization exceeds the system order, a transformation to the controller normal form is not possible. However, the system can be transformed to the *generalized controller normal form* and a *quasi-static feedback linearization* can be performed.

(P15) *Determine the controllability indices $(\kappa_1 \kappa_2)$ such that the resulting system of equations is locally regular with respect to $\mathbf{x}$. Specify the new inputs v_1 and v_2 and determine appropriate Hurwitz coefficients. Enter the Hurwitz coefficients into the script 'Initialization-Simulation.m'.*

- (P16) *Implement a quasi-static tracking control in the template Simulation.slx.*
- (P17) *Simulate the quasi-static tracking control with all desired trajectory generators for both the kinematic simulation model and the reference model. What differences from feedforward control can you identify? Again, save the plots of the actual and desired position of the load.*

Furthermore, a flatness-based tracking controller based on a dynamic feedback linearization is designed. For this purpose, the dynamic controller

$$\begin{aligned}\dot{\bar{x}}_1 &= \bar{x}_2 \\ \dot{\bar{x}}_2 &= \bar{u}_2 \\ u_1 &= \bar{u}_1 \\ u_2 &= \bar{x}_1 + x_3 x_6^2 - \bar{u}_1 \sin x_5\end{aligned}$$

is applied.

- (P18) *Show that the overall system of crane and dynamic controller is also flat.*
- (P19) *Implement a dynamic tracking control in the template Simulation.slx. Specify appropriate Hurwitz coefficients. Enter the Hurwitz coefficients in the 'Initialisierung-Simulation.m' script.*
- (P20) *Simulate the dynamic tracking control with all desired trajectory generators for both the kinematic simulation model and the reference model. What differences to feedforward control and quasi-static tracking control can you identify? Again, save the plots of the actual and desired position of the load.*
- (P21) *Discuss the simulation results with the supervisor of the experiment before you start running the experiment.*

3 Experiments

After the simulation with MATLAB/Simulink, the designed control is now tested on the experimental setup. For this purpose, contact the supervisor of the experiment in advance. Open MATLAB on the experimental computer. If a prompt for the dSpace system appears, select "rti1104".

- (E1) *Enter the coefficients of the parabola from (P8) into 'Initialization.m'. Then execute the script.*

In addition to the control, the Simulink model for the experimental setup also contains other blocks for the sensors and the drive. First, a setpoint change is implemented using the feedforward control from (P5).

- (E2) *Include the target trajectory generator for a setpoint change and feedforward control in the Simulink model.*

In order to run the code on the dSPACE system, it must first be compiled. Use *Build* (Ctrl-B) to compile the code and load it onto the dSPACE system. It also creates a *variable description* file (.sdf) in the current directory, which is needed in the following step. Once this is done successfully, open the dSPACE ControlDesk program and the ControlDesk project.

An interface for the measurement data is available in ControlDesk. With *Start Measuring* you start the recording of the measurement. Before you load a new version of your Simulink model, you must click *Go Offline*, otherwise an error message will appear during the build process. In the ControlDesk interface, you can specify the start and end points and the duration T of the desired trajectory.

- (E3) *Familiarize yourself with the operation of the interface. Enable the motor and then start the calibration.*
- (E4) *Test your feedforward control for setpoint changes with different start and end points and for $T = 3$. Do you always reach your target?*
- (E5) *Implement the other two desired trajectory generators and test them in combination with the feedforward control. Does the load follow the defined trajectories? Are the points of the setpoint change sequence traversed exactly?*
- (E6) *Implement the quasi-static and the dynamic tracking control. Compare the control performance of the two controls for different desired trajectories. What advantage do the controls offer compared to feedforward control?*
- (E7) *Try to make the system more dynamic and run the trajectory with $T = 2$ s. Does a higher speed have an effect on the control performance? What could cause possible changes in the system behavior?*

Lastly, a controller is developed that stabilizes the current setpoint. Because of the safety fence, it is not possible for you to introduce disturbances directly. For this reason, please contact the supervisor of the experiment to test this controller.

- (E8) *Implement a desired trajectory generator that stabilizes the current operating point. Test this together with the supervisor of the experiment.*

References

- [1] K. Graichen. Nonlinear Control Systems. *Lecture notes, Friedrich-Alexander Universität Erlangen-Nürnberg*, 2023.
- [2] T. Moor. Modellbildung in der Regelungstechnik. *Vorlesungsskript Friedrich-Alexander Universität Erlangen-Nürnberg*, 2021.